

```

++val;
futex_wake (&val, INT_MAX);
}
void ev_wait () {
    futex_wait (&val, val);
}
private:
    int val;
};

```

Here, the consumer threads call `ev_wait()` and pass in the current value of `val` (which is the same as what is stored at `&val`); hence, it is blocked. Note the semantics of the `futex_wait()` call above—although the interface shown here is slightly different. The producer thread calls `ev_signal()`, which increments the value of `val` and calls `futex_wake()` to wake up all the waiting threads (all since `INT_MAX` is passed as `val`; again, a different interface is shown here, for the sake of discussion).

Note here that incrementing `val` before calling `futex_wake()` is not really necessary, but here it is done to avoid a race condition. Let us suppose a new consumer thread had read the old value of `val` and was context-switched before calling `futex_wait()`. Now, after the producer called `ev_signal()`, if the `futex_wait()` is invoked in the new consumer, it would not block, since the value of `val` is not the same as what it had originally read.

2) **Implementing mutual exclusion (mutexes):** The following code illustrates one simple, trivial way to implement mutual exclusion using the above `futex` interface:

```

class mutex {
public:
    mutex () : val (0) { }
    void lock () {
        int c;
        while ((c = atomic_inc (val)) != 0)
            futex_wait (&val, c + 1);
    }
    void unlock () {
        val = 0;
        futex_wake (&val, 1);
    }
private:
    int val;
};

```

Here, a zero value for `val` represents an unlocked mutex. Any other value represents a locked one. It is easy to see how this provides mutual exclusion. When two different threads call `lock()` at the same time, only one of them finds that the old value was non-zero, and only that thread is put to sleep. Hence, only one thread enters the

critical section. In `unlock()`, only one thread is woken up at a time. This is kind of an optimisation, which works just fine with respect to correctness.

Although this seems to work, it has several problems.

The correctness problem (1): Consider the following scenario. The lock is taken by thread *T1*, and *T2* and *T3* execute `lock()` almost at the same time. Let us suppose the `lock()` code gets executed in the following manner:

Initially, assume `val` is 1;

```

T2
T3
lock()
atomic_inc (val) //now val=2, c=1          lock()

atomic_inc (val) //val=3, c=2
futex_wait (&val, c + 1); //value(val)=3
atomic_inc (val) //now val=4, c=3

futex_wait (&val, c + 1); //value(val)=4

```

In the above sequence, both `futex_wait` calls fail, since the actual value of `val` (3 and 4 as shown in the code) is not the same as what is being passed as second arg (`c+1 = 2` and `3` respectively). This sequence may repeat forever, making the program incorrect.

The correctness problem (2): In case of interrupts, a single thread may keep getting interrupted exactly when it calls `futex_wait()`. This system call returns whenever an interrupt occurs. Hence, a single thread may keep incrementing `val` until it overflows and becomes 0.

A performance-related problem: The `futex_wake()` call can be avoided in `unlock()` in the uncontended case.

Better mutex implementation

Consider the following revised implementation, which fixes these issues:

```

class mutex2 {
public:
    mutex () : val (0) { }
    void lock () {
        int c;
        if ((c = cmpxchg (val, 0, 1)) != 0)
            do {
                if (c == 2 || cmpxchg (val, 1, 2) != 0)
                    futex_wait (&val, 2);
            } while ((c = cmpxchg (val, 0, 2)) != 0);    ///
    }
    void unlock () {
        if (atomic_dec (val) != 1) {
            val = 0;
            futex_wake (&val, 1);
        }
    }
};

```